

AI-Powered National Skill Demand Intelligence & Planning Platform

Labour Market Intelligence, Skill Demand/Supply Analysis & Government Training Planning System

Industry → Jobs → AI / NLP → Skills → Skill Knowledge Graph → Demand Intelligence → Training Supply → Curriculum → Candidates → Jobs → Placement → Employer Feedback → AI Learning — **a closed loop, not a one-way pipeline.**

Executive Summary

Why this platform exists, and what it decides

This document is the complete technical and system-design blueprint for the **National Skill Demand Intelligence & Planning Platform**. It is **not** a job portal, a course-recommendation website, an AI chatbot, or a single dashboard. It is a continuously learning **closed-loop labour market intelligence system** that converts real industry hiring signals into concrete government decisions about what to teach, where to teach it, how many people to train, and how to correct course when outcomes fall short.



What the platform decides, on the government's behalf

Curriculum - What skills should be taught - Which courses should be updated	Capacity - How many candidates should be trained - Where training should be provided	Resourcing - How many trainers are required - What equipment is required
Market signal - Where skill shortages exist - Where training oversupply exists	Candidate guidance How candidates should prepare for jobs	Learning loop How placement outcomes feed back into the system

Document scope

The document proceeds from business problem to full technical architecture: user panels and roles, database design, APIs, AI/NLP architecture, end-to-end workflows, security, deployment, analytics, forecasting, recommendation logic, and a phased

implementation roadmap to national scale.

Source of truth. This blueprint is built directly from the platform's approved concept brief. Every core concept — Skill Demand Intelligence, Demand/Supply Mismatch, Job Posting Intelligence, Skill Extraction & Normalization, Skill Ontology, Skill Knowledge Graph, Course→Skill Mapping, Curriculum Gap, Employer Validation, Placement Feedback Loop, District-Level AI Planning, Demand Score, Future Demand Prediction, Role-Based Dashboards, Closed-Loop Labour Market Intelligence and the Skill Demand Digital Twin — is preserved and expanded below. Any figures shown as examples (e.g. district demand numbers) are clearly labelled as illustrative simulations, not guaranteed real-world values.

Why the Current System Fails

The demand/supply mismatch government training faces today

Why the current skill-development pipeline fails

Government training capacity today is planned largely on **historical enrolment patterns and static sector priorities**, not on what employers are actually hiring for right now. The result is a persistent, invisible mismatch between what training institutes produce and what industry needs.

Demand is invisible

- Thousands of job postings encode real-time skill demand, but it is unstructured text, never aggregated
- No one converts job descriptions into a measurable demand signal

Supply is disconnected

- Course curricula are updated on multi-year cycles
- Institutes rarely know which of their skills are still in demand

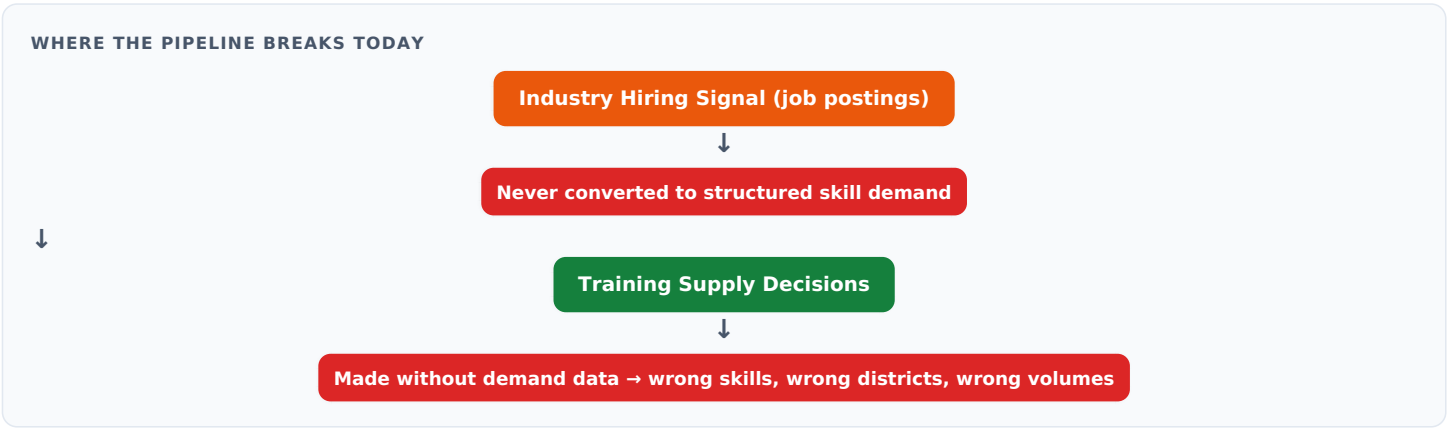
Planning is reactive

- District/state training targets are set without shortage or oversupply data
- Trainer and equipment gaps are discovered after batches under-perform

Feedback never closes the loop

- Placement outcomes and employer feedback rarely reach curriculum designers
- The same skill gaps recur year after year

Consequence: a structural demand/supply mismatch



What is required to fix it

- A system that continuously reads real job postings and extracts **structured, normalized skill demand**.
 - A live comparison between that demand and actual **training capacity, curriculum content, and trainer/equipment availability**.
 - A decision-support layer for **national, state and district government** that turns the gap into a concrete, explainable recommendation.
 - A feedback loop from **placement outcomes and employer validation** back into curriculum and capacity planning, so the system gets smarter every cycle.
- This is a data and decision-support problem before it is an AI problem. AI is the engine that makes the pipeline continuous and current — it does not replace human government decision-making, it informs it.

What We Are Building

A closed-loop intelligence platform, built on top of the existing Job Portal

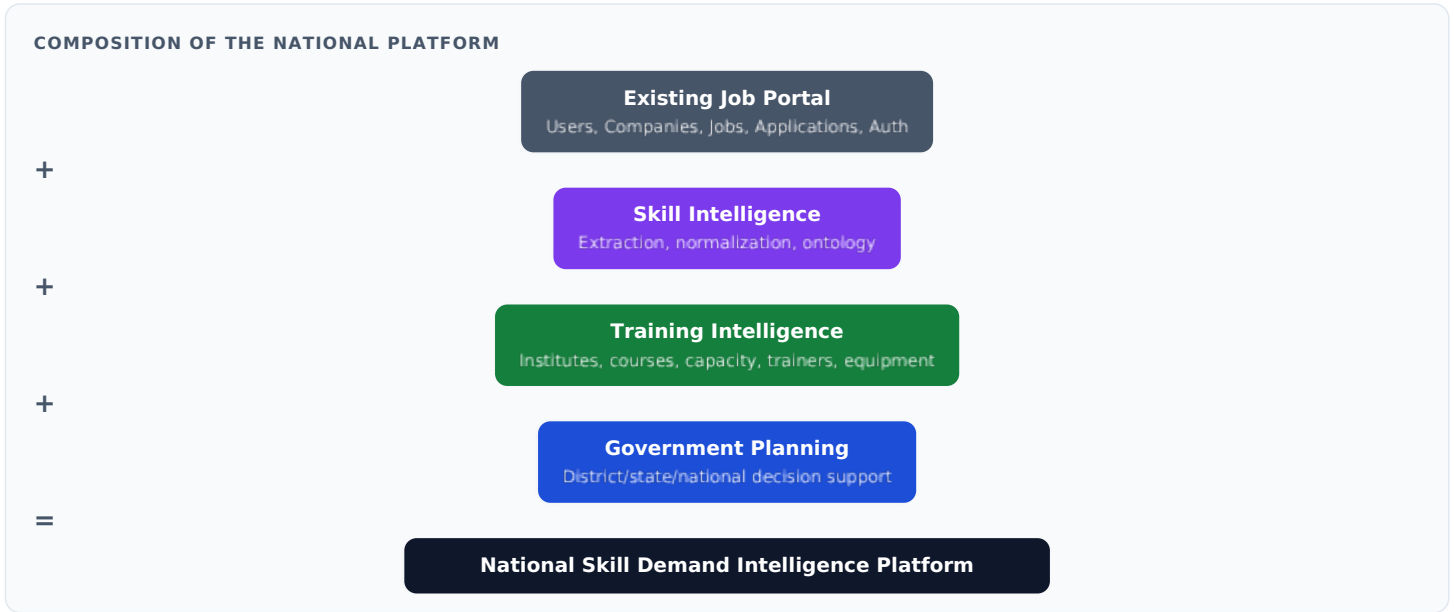
What we are building

A single, role-based platform that sits across the entire labour-market lifecycle: it reads industry demand, understands it with AI/NLP, compares it against training supply, recommends curriculum and capacity changes, guides candidates, and learns from real placement outcomes.

This is explicitly not: a job portal alone · a course-recommendation site alone · a chatbot · a single dashboard. Those are components used inside a much larger closed-loop intelligence system.

Core architecture decision — extend, don't replace

The organisation already operates a **Job Portal** with Users/Candidates, Companies/Employers, Jobs, and Job Applications. That system is the **base layer** of the new platform, not a separate product.



Why reuse the existing Job Portal instead of building a new one

REASON	IMPACT
No duplicate identity/data model	Users, Companies and Jobs already have authentication, profiles and relationships — rebuilding them risks two sources of truth for the same candidate or employer.
Immediate real data	Live job postings and applications already flow through the portal, giving the intelligence layer real signal from day one instead of a cold start.
Faster time to MVP	Engineering effort goes into the new intelligence services, not re-implementing auth, profiles and job posting.
Lower operational risk	The live job-seeking and hiring workflow that companies and candidates already depend on is never disrupted.
Natural extension points	New collections (Skill, JobSkill, CandidateSkill, etc.) simply reference existing User, Company and Job documents.

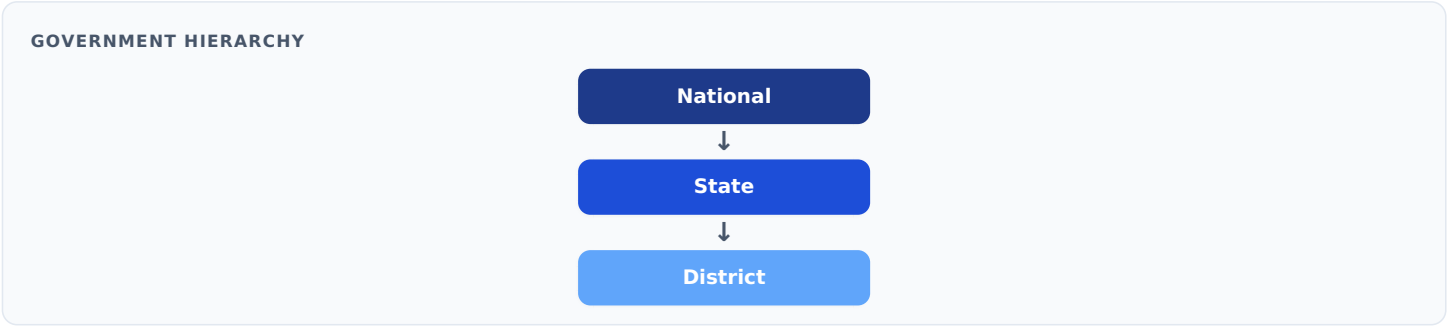
Who Uses the Platform

Eight roles, one shared data spine, zero overlapping responsibility

The platform serves eight categories of user, each with a distinct panel, distinct responsibilities, and a distinct slice of the data. No panel duplicates another panel's decisions.

Super Admin Owns the platform itself: ontology, roles, configuration, AI monitoring	National Government Sees the whole country; sets national training policy	State / Regional Government Compares districts within a state; allocates budget
District Government Makes local training decisions for its district	Training Institute Runs courses, batches, trainers and equipment	Employer / Company Source of demand; validates skills and curriculum
Student / Candidate Learns, closes skill gaps, gets placed	Public Views non-sensitive labour-market insights	

Hierarchical permission structure



A national user can see everything below it; a state user can see only its own districts; a district user can see only itself. Super Admin sits outside this chain and manages the platform, not day-to-day planning decisions.

Super Admin Panel

Platform governance, ontology control, AI & data-quality oversight

Role: Platform-level administrator. **Purpose:** manage the platform itself — ontology, configuration, access, data quality — rather than making day-to-day district skill-planning decisions.

Platform governance • Manage system & government administrators • Manage roles and permissions • Manage districts • Audit logs / system health	Skill ontology control • Manage master skills • Manage skill aliases • Approve unknown AI-detected skills	Master data & config • Manage master data • Manage system configurations • Platform-level settings
AI & data quality oversight • Monitor AI processing • Monitor data quality • User & access control		

Pages inside this panel

Overview · Users · Roles & Permissions · Government Admins · Master Skills · Skill Aliases · Roles · Districts · System Settings · AI Processing Monitor · Audit Logs · System Health

Super Admin never edits a district's training plan — it edits the rules the whole platform runs on (who can do what, which skills exist, how AI output is governed).

Government / National Admin Panel

National labour-market intelligence and training-planning decisions

Role: Government decision-support user. **Purpose:** understand national/state/district labour-market conditions and turn them into training-planning decisions.

Dashboard • Total Jobs Analysed • High-Demand Skills • Critical Skill Gaps • Training Capacity • Trainer / Equipment Gaps • Placement Rate • Emerging Skills	District Intelligence • District demand & supply • Skill shortage / oversupply • Industry demand • Training capacity, trainers, equipment	Skill Demand • By skill, role, district, industry • Demand growth & demand score
Forecast • Historical demand • Future demand prediction • Emerging skills & technology trends	Supply / Demand • Demand vs. training capacity • Gap, shortage, oversupply	Curriculum Intelligence • Industry skills vs. course skills • Missing skills & recommendations
Training Planning • Required batches, seats, trainers, equipment	AI Recommendations • Increase/reduce training • Add modules • More practical training • Arrange trainers, procure equipment	What-If Simulator • e.g. “What happens if 500 additional EV Technicians are trained?”
Reports • District, skill, training, placement, forecast reports		

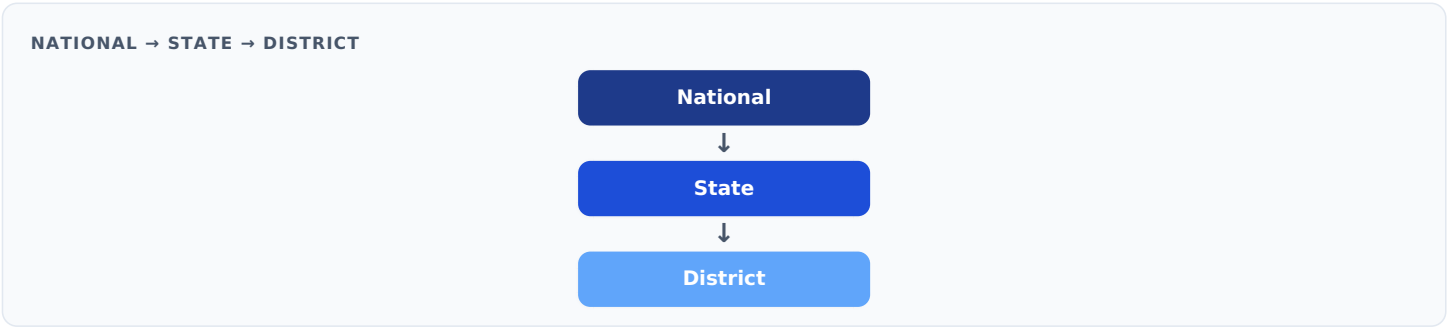
State / Regional Government Admin

Cross-district comparison and regional resource planning

Role: State or regional authority. **Purpose:** analyze and plan skill development across multiple districts within the state.

Comparative view - State overview - District comparison & ranking	Maps - Skill demand map - Skill shortage map	Capacity - Training capacity comparison - Course performance across districts
Planning - State-wide skill gaps - Regional forecasts - Budget / resource planning - Training allocation recommendations		

Hierarchical permission model



A State user can drill into every district that belongs to their state, but cannot see other states' internal district data unless explicitly granted — identical role/permission engine as Super Admin's Roles & Permissions module, scoped by stateId.

District Admin Panel

Local training decisions, grounded in local demand data

Role: District-level government/planning authority. **Purpose:** make local training decisions.

District dashboard

- Local job demand
- Local skill demand
- Local shortages & oversupply

Local resources

- Institutes, courses, capacity
- Trainers, equipment

Outcomes

- Placement performance
- Local employer demand

Decisions

- AI recommendations
- District training plan

Worked example (illustrative simulation, not a live figure)

NAGPUR · EV TECHNICIAN	
METRIC	VALUE
Demand	850
Capacity	200
Gap	SHORTAGE · 650

AI recommendation for this example

- Increase seats
- Add batches
- Train additional trainers
- Procure equipment

All numeric examples in this document (Nagpur EV Technician, the React/Node skill profile, the 500-EV-Technician simulation, etc.) are **illustrative** — they demonstrate how the engine reasons, not guaranteed real-world values.

Training Institute Panel

Turning industry demand into curriculum and capacity decisions

Role: Training provider. Institutes need one question answered continuously: **“What should we teach based on actual industry demand?”**

Operations • Institute dashboard • Courses, course modules, curriculum • Batches, seats, capacity	People & equipment • Students, trainers • Equipment • Attendance / progress if applicable	Outcomes • Placement • Employer feedback
Intelligence • Curriculum gap • Skill demand relevant to this institute • AI recommendations		

Employer / Company Panel

Where real demand enters the system and gets validated

Role: Industry demand source and validation authority. The existing Company/Employer system is reused — this panel adds skill-intelligence features on top of it.

Demand

- Company dashboard
- Job posting
- Required skills & proficiency
- Demand submission

Validation

- Validate skills
- Validate curriculum

Talent

- Find trained candidates
- Candidate feedback

Outcomes

- Interview outcome
- Placement feedback

Worked example — skill-proficiency submission

JUNIOR FULL STACK DEVELOPER · REQUIRED PROFICIENCY (ILLUSTRATIVE)	
SKILL	REQUIRED PROFICIENCY
React	5 / 5
Node.js	4 / 5
Git	4 / 5
Docker	3 / 5
AWS	3 / 5

This structured proficiency data is what curriculum comparison and candidate skill-gap analysis are measured against — it is far more useful than a free-text job description alone.

Student / Candidate Panel

Skill assessment, gap analysis and a guided path to a job

Role: Individual learner / job seeker. The existing User/Candidate system is reused and extended with skill and career-path intelligence.

Profile

- Profile, education
- Skills, skill assessment
- Current proficiency

Guidance

- Target career
- Skill gap
- Career recommendation & pathway

Action

- Recommended courses
- Local skill demand
- Jobs, applications

Tracking

- Progress
- Placement

Worked example — skill gap and pathway (illustrative)

CANDIDATE TARGETING: FULL STACK DEVELOPER

SKILL	STATUS
HTML	HAVE
CSS	HAVE
JavaScript	PARTIAL
React	MISSING
Node.js	MISSING
Git	MISSING

RECOMMENDED LEARNING PATH

JavaScript

↓

React

↓

Node.js

↓

Git

↓

Projects

↓

Job

Analytics / Public Intelligence Portal

Public, non-sensitive labour-market visibility

Role: Optional public-facing layer. **Purpose:** provide non-sensitive labour-market insight to the public without exposing private candidate or employer data.

Public insight • Skill demand trends • Emerging skills • District demand	Career context • Career demand • Skill rankings	Transparency • Public reports
---	--	--

Data boundary: this panel only ever reads aggregated, anonymised DemandSnapshot and Recommendation-summary data. It has no access to CandidateSkill, EmployerFeedback, or any personally identifiable record.

Panel Role Matrix

One permission model, enforced consistently across every panel

One consolidated permission matrix across every panel. **V**=View, **C**=Create, **E**=Edit, **A**=Approve, **M**=Manage (full control), **N**=Analyze.

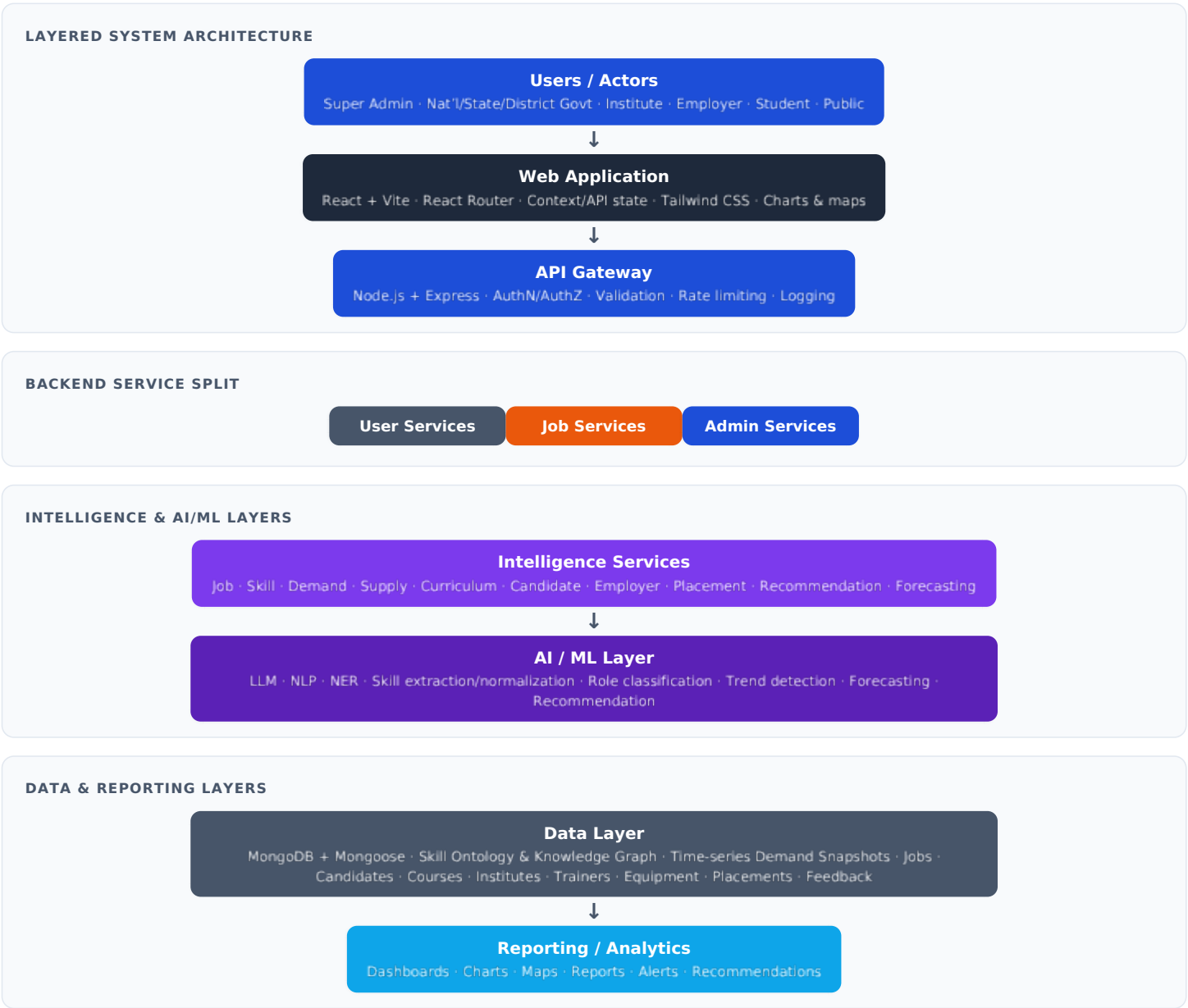
View (V) Create (C) Edit (E) Approve (A) Manage (M) Analyze (N)

Feature	Super Admin	National Govt	State Govt	District Govt	Institute	Employer	Student
User Management	M	V	V	V	—	—	—
Skill Master	M	V E	V	V	V	V	V
District Management	M	M	V	V	—	—	—
Job Demand	V	V N	V N	V N	V	C	V
Skill Demand	V	V N	V N	V N	V	V	V
Supply	V	V N	V N	V N	C	—	—
Curriculum	V	V N	V	V	C E	A	V
Training Capacity	V	V N	V N	V N	C E	—	—
Trainer Planning	V	V	V	V M	C E	—	—
Equipment Planning	V	V	V	V M	C E	—	—
Forecast	V	V N	V N	V N	V	V	—
AI Recommendations	V	V N	V N	V N	V	—	V
What-If Simulation	V	M	M	M	—	—	—
Employer Validation	V	V	V	V	V	A	—
Candidate Skill Gap	V	V N	V N	V N	V	V	V
Placement	V	V N	V N	V N	C	C V	V
Feedback	V	V	V	V	V	C	V
Reports	V	C	C	C	C	V	V

Manage (M) implies View, Create and Edit for that feature within the user's scope (national / state / district). Approve (A) is reserved for workflows that require a human decision on AI output (e.g. an employer approving a validated skill list, or an unknown skill needing Super Admin approval).

Complete System Architecture

Six layers: users, web app, gateway, services, AI, data & reporting

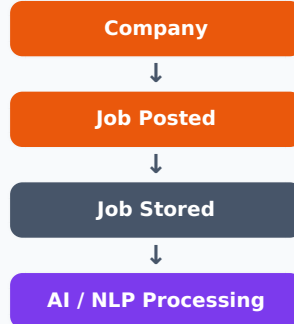


From Job Posting to System Learning

The complete lifecycle of one unit of demand, traced through every stage

This is the single most important diagram in the document: it traces one job posting from the moment a company creates it to the moment its outcome improves the system's future recommendations.

STAGE 1 • INGESTION & EXTRACTION



STAGE 1A • WHAT AI EXTRACTS FROM ONE POSTING



STAGE 2 • NORMALIZATION & AGGREGATION



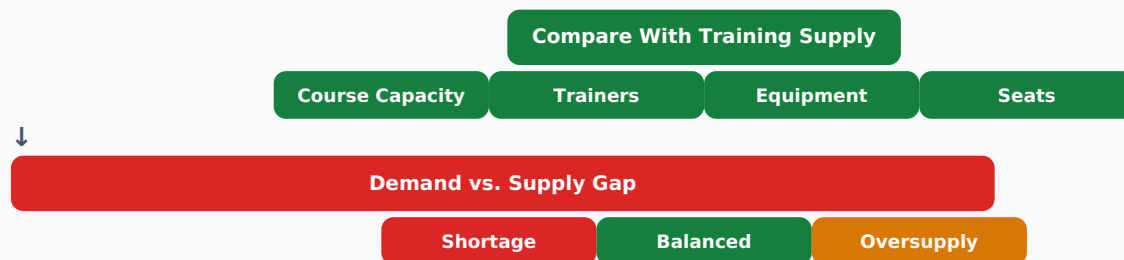
STAGE 2A • DEMAND IS AGGREGATED ALONG FOUR AXES



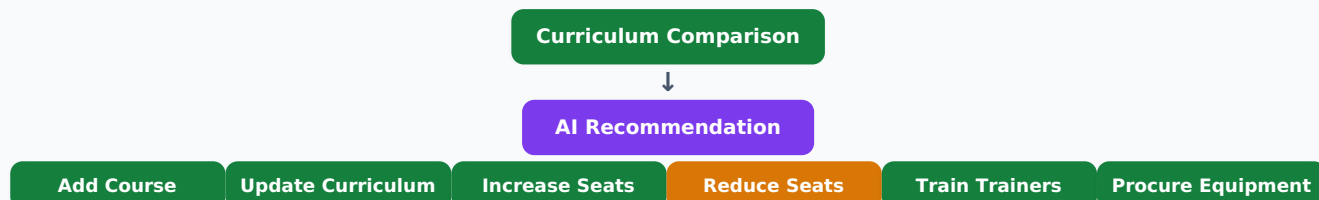
STAGE 3 • SCORING & FORECASTING



STAGE 4 · COMPARE AGAINST TRAINING SUPPLY



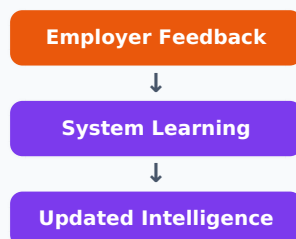
STAGE 5 · CURRICULUM COMPARISON & RECOMMENDATION



STAGE 6 · CANDIDATE PATH TO PLACEMENT



STAGE 7 · CLOSING THE LOOP



Notice the loop: “Updated Intelligence” feeds directly back into Demand Aggregation, Forecasting and Curriculum Comparison for the next cycle — this is what makes the platform a *closed-loop* system rather than a one-time report generator.

AI Is Not a Chatbot

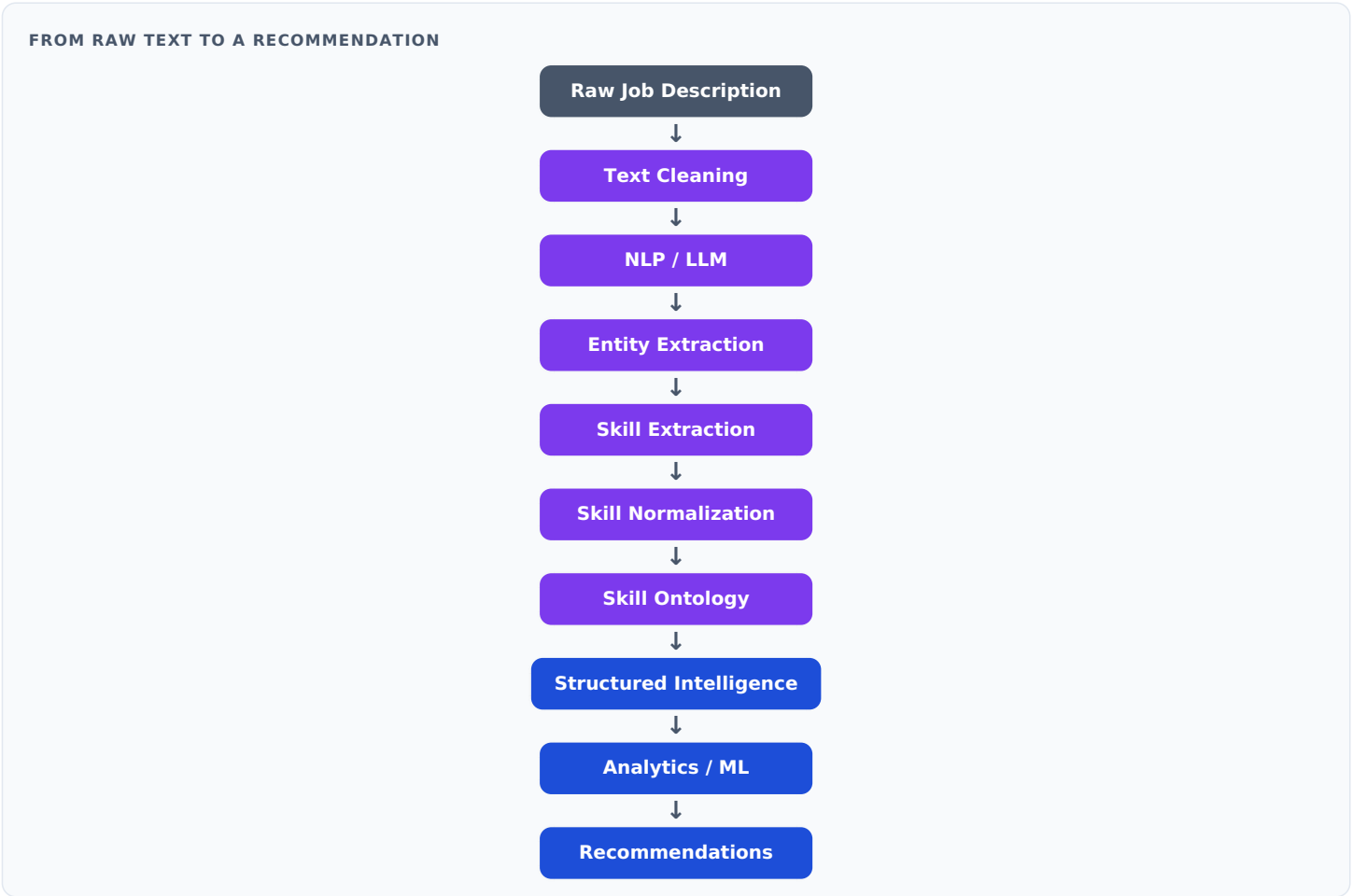
Twelve specialised jobs that structure demand, not one conversational feature

AI in this platform is not a chatbot. It is a set of twelve specialised, mostly-invisible processing jobs that run on data as it moves through the system.

The twelve AI responsibilities

#	RESPONSIBILITY	RUNS ON
1	Job Description Parsing	Every new job posting
2	Skill Extraction	Job description text
3	Skill Normalization	Extracted raw skill terms
4	Role Classification	Job title + description
5	Skill Relationship Detection	Skill co-occurrence across jobs
6	Trend Detection	Time-series demand snapshots
7	Emerging Skill Detection	Newly-seen, fast-growing skill terms
8	Skill Gap Analysis	Candidate skills vs. target role
9	Curriculum Recommendation	Course skills vs. industry skills
10	Demand Forecasting	Historical demand snapshots
11	Training Recommendation	Demand/supply gap
12	District Planning Recommendation	Aggregated district intelligence

AI processing pipeline



Every AI stage produces **structured, typed JSON**, never free text. That structured output is what the rest of the platform (dashboards, matching, forecasting) is built on — the model is a structuring engine, not a conversational surface.

Technology Stack

Reuse what exists; add intelligence as modular services, not microservices

Current base system (already in production)

LAYER	TECHNOLOGY
Frontend	React · Vite · React Router · Context API · Tailwind CSS
Backend	Node.js · Express.js · REST APIs
Database	MongoDB · Mongoose
Authentication	Clerk (candidates/users) · JWT + bcrypt (employers/companies)
Storage	Cloudinary
Monitoring	Sentry

Intelligence layer

PHASE	TECHNOLOGY
MVP	Node.js AI service · LLM API · structured JSON output · rule-based normalization · MongoDB
Future / scale-out	Python · FastAPI · NLP libraries · ML forecasting · dedicated knowledge-graph technology if required

Do not introduce microservices at MVP. A modular, well-organised Node.js monolith (clear service boundaries inside one deployable app) is faster to build, easier to debug, and cheap to operate at pilot scale. Microservices add network overhead, deployment complexity, and operational cost that only pay off once traffic, team size, or independent scaling needs justify them — none of which apply at MVP.

Database Architecture

Twenty new collections extend four existing ones — nothing is duplicated

Existing entities (reused, not duplicated)



New entities introduced by the intelligence platform

ENTITY	PURPOSE
Skill	Canonical, normalized skill record
SkillRelationship	Skill-subskill / co-occurrence edges (the knowledge graph)
Role	Job role mapped to a required-skill profile
JobSkill	Skill required by a specific Job, with weight/proficiency
JobIntelligence	AI extraction result for one Job (role, skills, location, experience, salary)
CandidateSkill	Skill a candidate holds, with proficiency level
TrainingInstitute	Provider offering courses in a district
Course	Program offered by an institute
CourseModule	Unit within a course
CourseSkill	Skill taught by a course/module
TrainingBatch	A scheduled run of a course with seats and dates
Trainer	Staff qualified to teach specific skills
Equipment	Physical/lab resource required to teach a skill
District	Geographic/administrative planning unit
EmployerFeedback	Company's rating of a candidate/placement outcome
Placement	Candidate placed into a Job at a Company
DemandSnapshot	Time-series aggregation of demand by district/skill/role
Forecast	Predicted future value of a DemandSnapshot metric
Recommendation	AI-generated action tied to a District/Skill/Course
AuditLog	Immutable record of administrative actions

Key relationships

CHAIN
Company → Job → JobSkill → Skill
Role → Skill
Course → CourseModule → CourseSkill → Skill
User → CandidateSkill → Skill
TrainingInstitute → Course → TrainingBatch → Trainer → Equipment
District → Jobs → Institutes → DemandSnapshot
Placement → User → Company → Job
EmployerFeedback → Company → Candidate / Placement
DemandSnapshot → District → Skill / Role
Recommendation → District / Skill / Course

Every new collection references existing User, Company or Job documents by ObjectId — nothing about the base Job Portal schema is forked or copied.

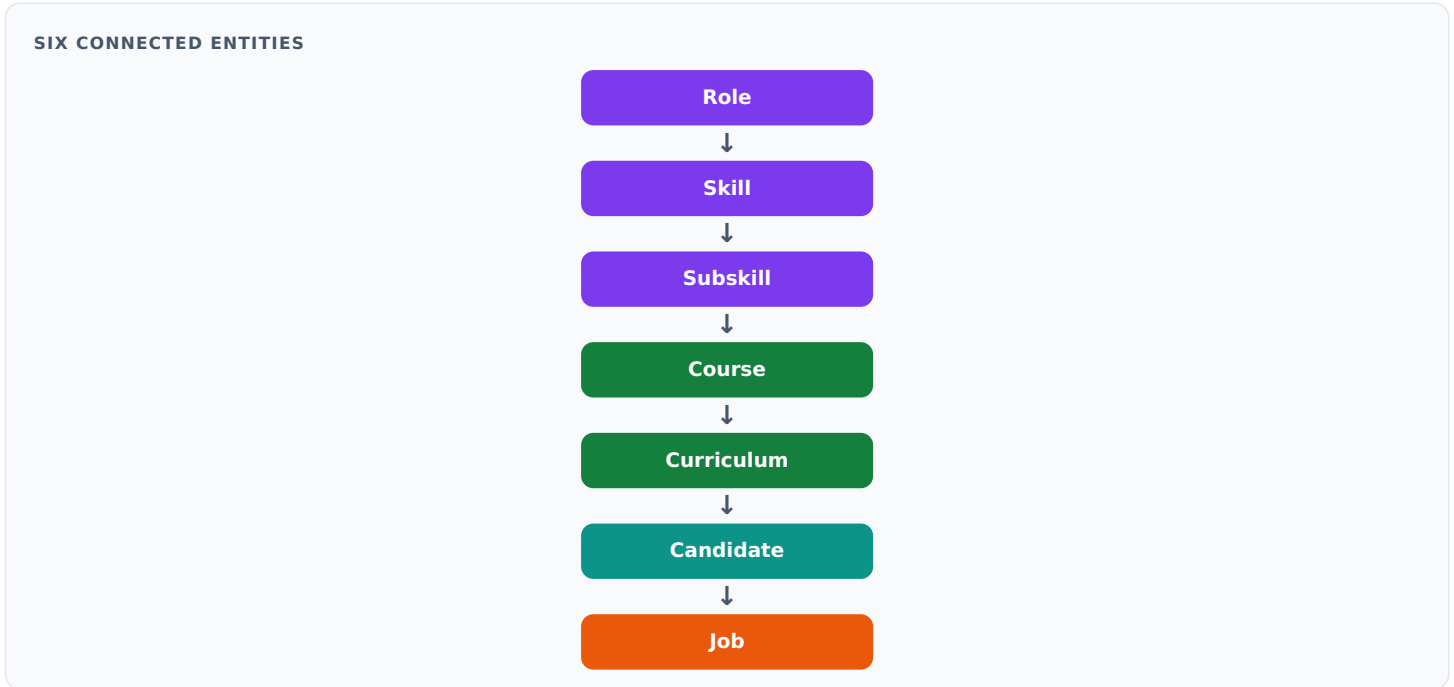
Skill Knowledge Graph

Role ↔ Skill ↔ Subskill ↔ Course ↔ Curriculum ↔ Candidate ↔ Job

The knowledge graph is what lets the platform reason about skills the way a human curriculum designer does — as a hierarchy of related capabilities, not a flat tag list.

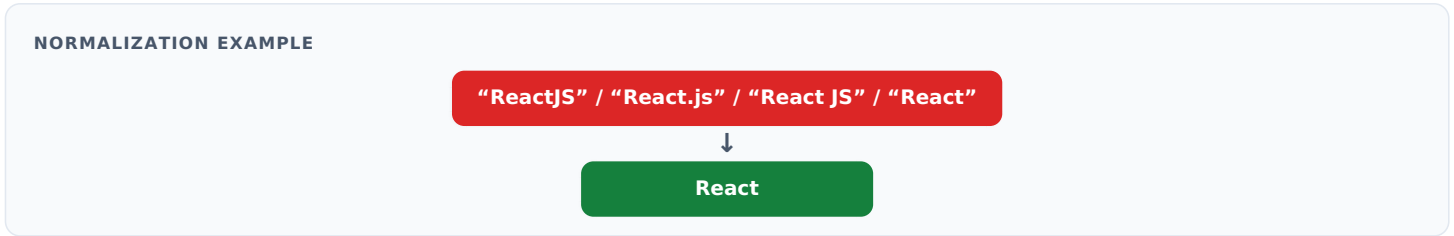


Graph relationships the platform maintains



Why normalization and ontology are mandatory

Free-text job postings and resumes refer to the same skill in many different ways. Without a controlled ontology, demand for one skill silently fragments into several “different” skills and every downstream number — demand score, gap, forecast — becomes wrong.



Unknown terms detected by AI do not automatically become new Skill records — they enter a Super Admin approval queue first (see Panel 1 and Error Handling & Reliability), keeping the ontology controlled and trustworthy.

The Demand/Supply Engine

Turning raw signals into one comparable Demand Score and a clear gap status

Demand score calculation



Skill gap calculation

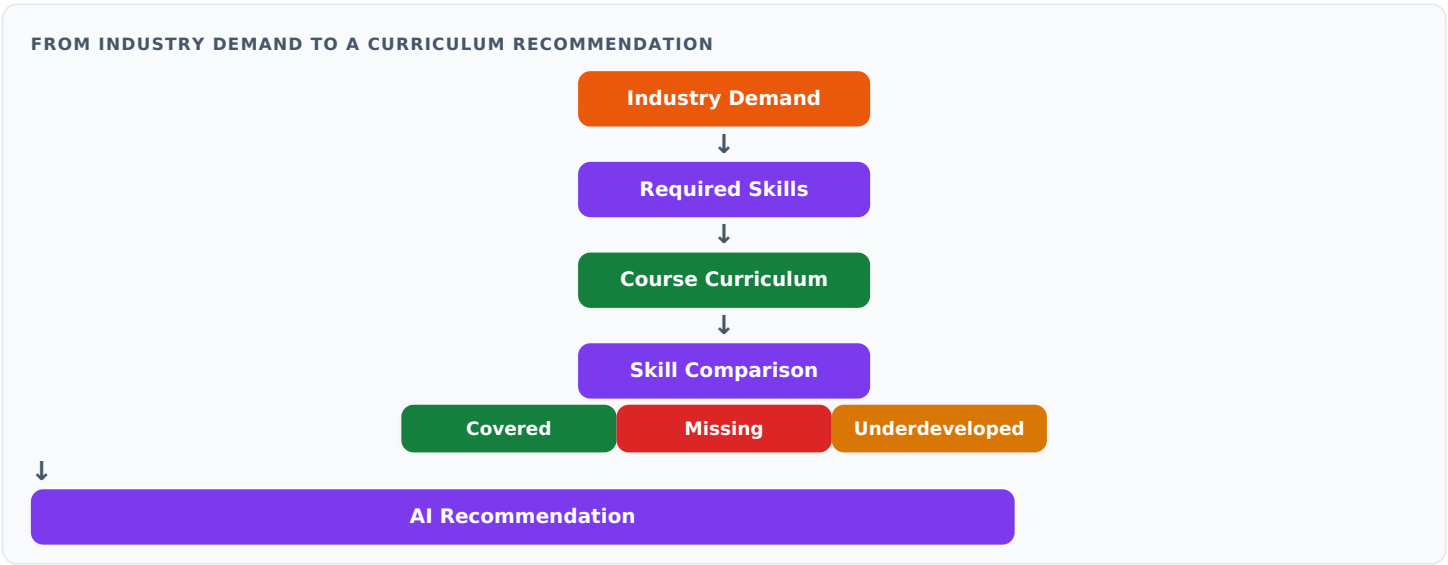


GAP VALUE	STATUS
Positive gap	SHORTAGE
Near zero	BALANCED
Negative gap	OVERSUPPLY

The exact scoring weights above are **configurable, not fixed**. They must be tuned and validated against real placement and hiring outcomes before being treated as authoritative — the architecture treats them as a parameterised model, editable from Super Admin → System Settings.

Curriculum Intelligence

Comparing what industry needs against what a course actually teaches



Worked example (illustrative)

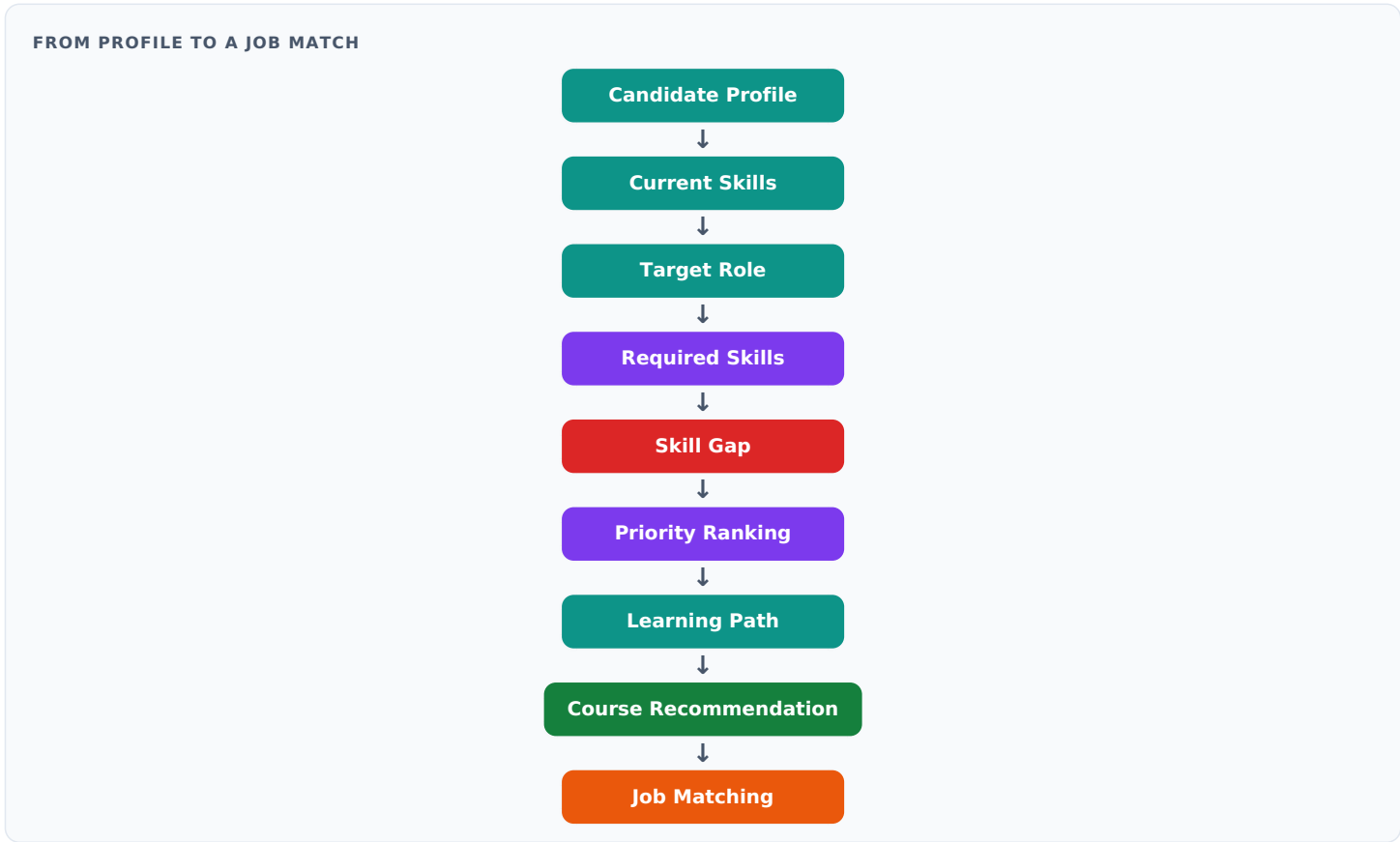
SKILL	INDUSTRY REQUIRES	CURRICULUM COVERS
React	✓	✓
Node	✓	✓
Git	✓	✗
Docker	✓	✗

Recommendation generated for this example

- Add Git
- Add Docker
- Increase practical React hours

Candidate Intelligence

A ranked, personalised path from current skills to a job match



Priority ranking matters: a candidate missing five skills should not be told to learn all five at once — the engine ranks the gap by demand weight and prerequisite order so the learning path is achievable.

Placement Feedback Loop

How real outcomes correct curriculum and capacity for the next cohort



Example metrics tracked



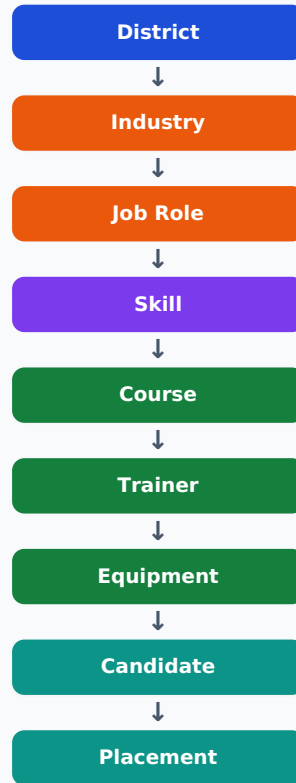
This is the loop that makes the platform genuinely closed: without Employer Feedback flowing back into Curriculum Recommendation, every earlier stage (extraction, scoring, forecasting) still just produces a one-time report.

Skill Demand Digital Twin

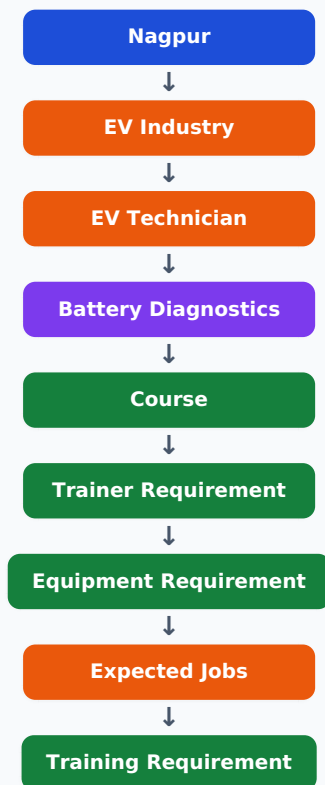
A live, queryable model of a district's entire training ecosystem

The **Skill Demand Digital Twin** is a live, structured representation of a district's entire labour ecosystem — industries, roles, skills, courses, trainers, equipment, candidates and placements — kept continuously in sync with real data so it can be queried and simulated against.

WHAT THE DIGITAL TWIN REPRESENTS, PER DISTRICT



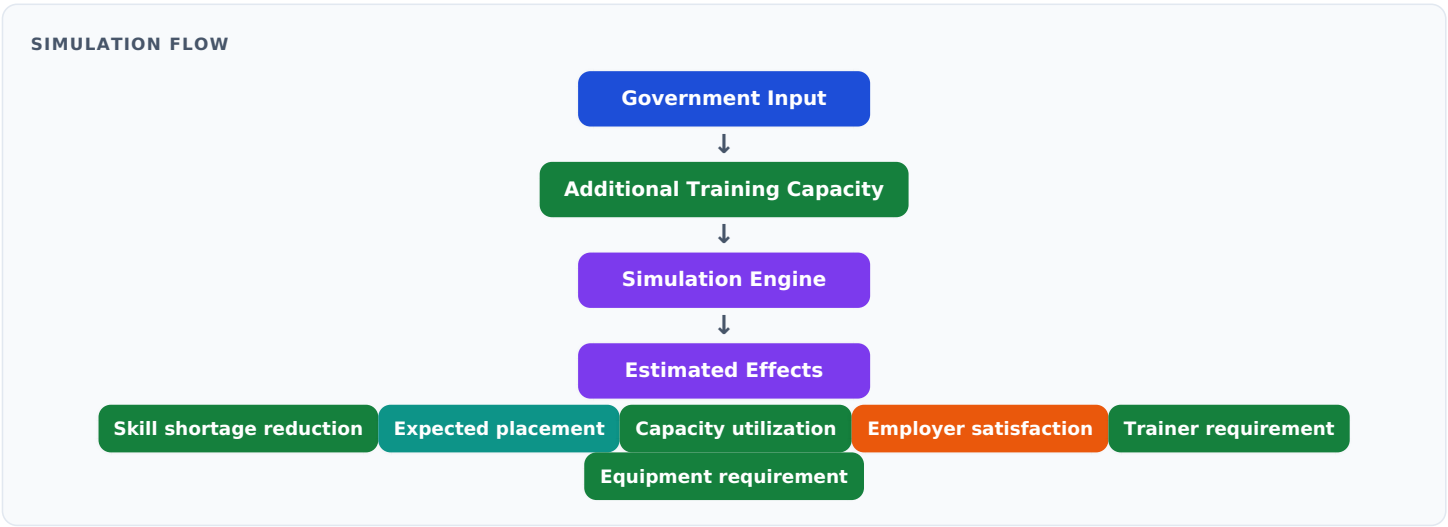
Worked example (illustrative simulation)



Because every node in the twin is backed by live database records rather than a static report, government can query it (“show every district where EV Technician demand exceeds capacity by more than 300”) and simulate against it (see the What-If Simulator, next page) instead of waiting for a manually compiled study.

What-If Simulator

Testing a policy decision against the digital twin before committing budget



Worked example

Input: “Train 500 additional EV Technicians.”

Output: estimated placement impact, estimated shortage reduction, estimated resource requirement.

Every simulator output is labelled **SIMULATED / PREDICTED / ESTIMATED** in the UI. These are model-driven projections to support a decision, not guaranteed actual outcomes — the platform must never present them as facts.

API Architecture

Grouped by domain, protected by the same role/scope model as the panels

API groupings

GROUP	BASE PATH
Core (existing Job Portal)	/api/auth, /api/jobs, /api/users, /api/companies
Government & platform admin	/api/admin, /api/government, /api/districts
Skills & intelligence	/api/skills, /api/intelligence, /api/demand, /api/supply
Curriculum, candidates, employers	/api/curriculum, /api/candidates, /api/employers
Outcomes & analytics	/api/placements, /api/analytics, /api/forecast, /api/recommendations
Planning	/api/simulator

Representative endpoints

METHOD & PATH	PURPOSE
POST /api/intelligence/parse-job	Run AI extraction on a job posting
GET /api/intelligence/demand/:district	District-level demand intelligence
GET /api/intelligence/emerging-skills	Newly detected, fast-growing skills
GET /api/demand/skills	Demand aggregated by skill
GET /api/demand/roles	Demand aggregated by role
GET /api/demand/trends	Time-series demand trend
POST /api/supply/institute	Register a training institute
GET /api/supply/institutes	List institutes
POST /api/supply/course	Create a course
GET /api/supply/capacity	Aggregate training capacity
GET /api/analytics/gap-analysis	Demand vs. supply gap by district/skill
GET /api/analytics/curriculum-gap	Curriculum vs. industry skill comparison
POST /api/analytics/simulate	Run a What-If simulation
GET /api/admin/dashboard	Aggregated admin dashboard data

Every route under /api/admin, /api/government, /api/districts, /api/supply, /api/analytics and /api/simulator requires an authenticated session and passes through role/scope authorization middleware before reaching a controller — a District user's JWT can only ever resolve data scoped to their own districtId.

Security Architecture

Role-based access, audit trails, and a hard public/private data boundary

Identity & access

- Authentication (Clerk / JWT+bcrypt)
- Authorization per route
- Role-based access control
- Hierarchical government permissions (National → State → District)

API protection

- Input validation on every endpoint
- Rate limiting
- Structured request logging

Accountability

- Audit logs for administrative actions
- Secure secrets management
- AI API key protection (never exposed client-side)

Data privacy

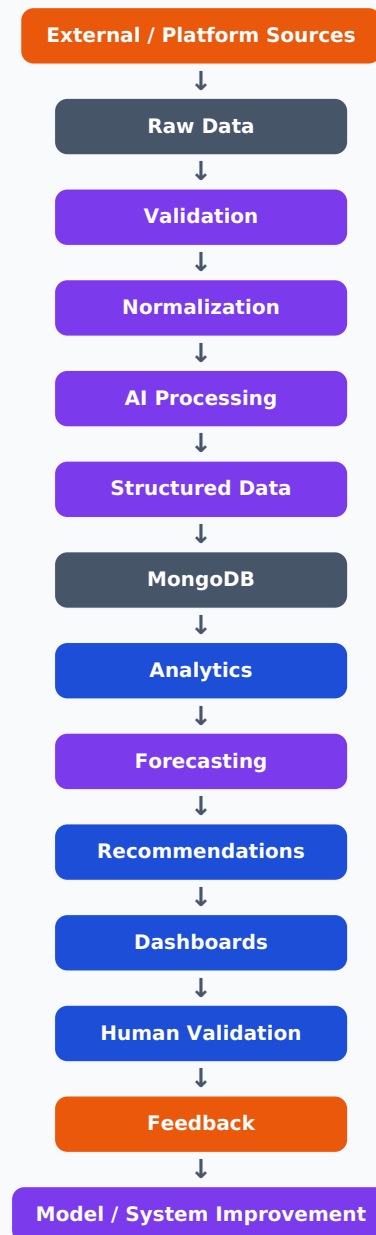
- Candidate data protection
- Employer data protection
- Strict public/private data separation (see Panel 8)

The Analytics / Public Intelligence Portal is the only surface exposed without authentication, and it is deliberately restricted to aggregated, anonymised data — it can never resolve back to an individual candidate, employer, or job application.

End-to-End Data Flow

Every input passes through validation and human review before it reaches decisions

FROM RAW INPUT TO A VALIDATED RECOMMENDATION



Human Validation is a first-class stage, not an afterthought: government users approving or rejecting a recommendation is itself feedback data that improves future scoring — the system is designed to learn from its administrators, not just from placement outcomes.

Implementation Roadmap

Phases 0–7: foundation, ontology, demand intelligence, training supply

Sixteen phases, each independently testable, each shippable before the next begins.

Phase 0 · Existing Job Portal Audit

Objective	Confirm the base system's schema, auth and API surface before extending it
Backend	Document existing models, routes, middleware
Frontend	Inventory existing pages/components to reuse
Database	Schema review of User, Company, Job, JobApplication
Dependencies	None — this phase gates everything else

Phase 1 · Admin/Government Authentication & Authorization

Objective	Add role-based auth for admin/government users on top of existing auth
Backend	Role model, JWT scope claims, permission middleware
Database	Add Role, and scope fields on admin users
Testing	Access-control tests per role
Output	A user can log in and be correctly scoped to national/state/district

Phase 2 · Admin Backend Foundation

Objective	Core CRUD services admin/government panels depend on
Backend	District, master-data, settings services
Database	District, AuditLog
APIs	/api/admin, /api/districts base routes
Output	Admin backend ready for a UI

Phase 3 · Admin Frontend/UI

Objective	Build the Super Admin and Government panel shells
Frontend	Sidebar/topbar layout, role-scoped navigation
Output	Navigable admin panels with placeholder data

Phase 4 · Skill Master

Objective	Establish the controlled skill ontology
Backend	Skill, SkillRelationship services
Database	Skill, SkillRelationship, alias handling
APIs	/api/skills
Output	A governed, de-duplicated skill list the rest of the system references

Phase 5 · Job Intelligence + AI Extraction

Objective	Turn job postings into structured intelligence
Backend	JobIntelligence service, async processing queue
AI work	LLM-based parsing, entity extraction
Database	JobIntelligence, JobSkill
APIs	/api/intelligence/parse-job
Dependencies	Phase 4 (Skill Master)

Phase 6 · Demand Intelligence

Objective	Aggregate structured job intelligence into demand metrics
Backend	Aggregation jobs, Demand Score calculation
Database	DemandSnapshot
APIs	/api/demand/*
Dependencies	Phase 5

Phase 7 · Training Supply

Objective	Model institutes, courses and capacity
Backend	Supply services
Database	TrainingInstitute, Course, TrainingBatch, Trainer, Equipment
APIs	/api/supply/*

Implementation Roadmap (continued)

Phases 8–15: curriculum, feedback, forecasting, simulation, digital twin

Phase 8 · Curriculum Intelligence

Objective Compare course content against industry-required skills
Backend CourseSkill mapping, comparison engine
APIs /api/analytics/curriculum-gap
Dependencies Phases 5-7

Phase 9 · Candidate Skill Gap

Objective Give candidates a personalised gap and learning path
Backend Skill-gap and ranking service
Database CandidateSkill
Dependencies Phase 4, 8

Phase 10 · Employer Validation

Objective Let employers confirm/correct AI-extracted skill demand
Backend Validation workflow + approval queue
Dependencies Phase 5

Phase 11 · Placement Feedback

Objective Capture outcomes and feed them back to curriculum/capacity
Database Placement, EmployerFeedback
Dependencies Phases 7-10

Phase 12 · Forecasting

Objective Predict future demand from historical snapshots
AI work Time-series forecasting models
Database Forecast
Dependencies Phase 6, sufficient historical data

Phase 13 · AI Recommendations

Objective Generate explainable training/curriculum recommendations
Database Recommendation
Dependencies Phases 6-12

Phase 14 · What-If Simulator

Objective Let government test policy scenarios against the digital twin
APIs /api/simulator
Dependencies Phase 13

Phase 15 · Digital Twin / Advanced Intelligence

Objective Full district-level digital twin and national-scale intelligence
Dependencies All prior phases; sufficient data maturity

MVP vs. Advanced System

What ships first, and what compounds on top of it

MVP

- Existing Job Portal (reused as-is)
- Admin panel (Super Admin + Government shell)
- Skill Master
- Job skill extraction
- Skill normalization
- Demand analytics
- Supply/demand gap
- Basic curriculum mapping
- Basic recommendations

Advanced

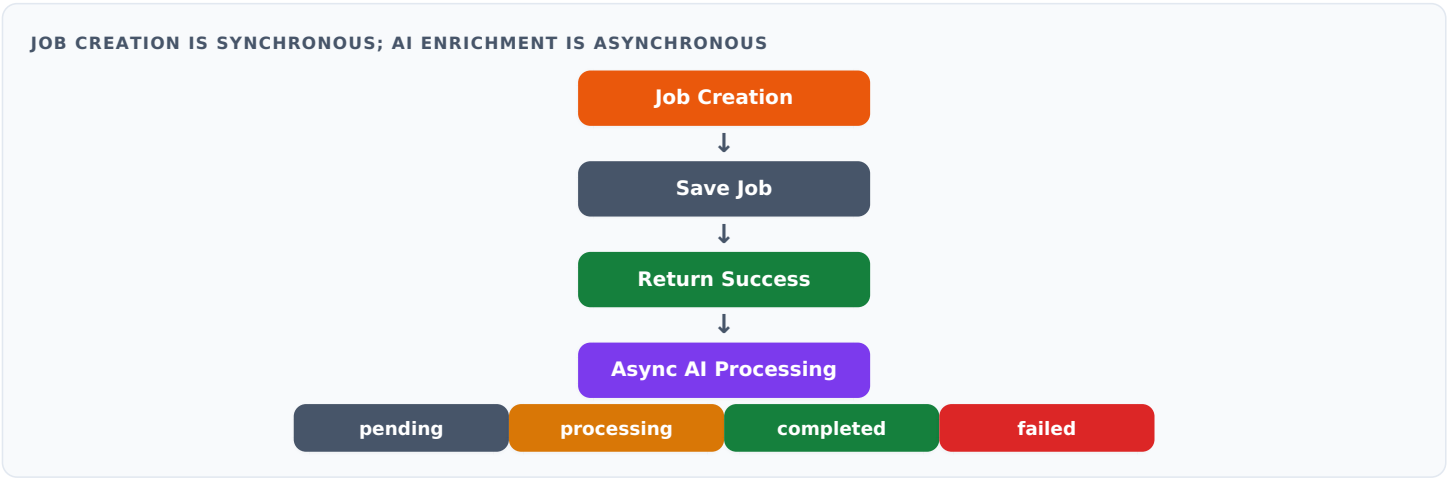
- Full skill knowledge graph
- Advanced forecasting
- Emerging technology detection
- Employer validation workflow
- Placement feedback loop
- District digital twin
- What-if simulation
- National-scale intelligence

The MVP column corresponds to Phases 0–8; the Advanced column corresponds to Phases 9–15. Nothing in the Advanced column is required to prove the core value proposition — it compounds value once the MVP is validated with real data.

Error Handling & Reliability

Core actions never depend on AI succeeding

Core principle: AI failure must never break job posting



- The company sees their job posted immediately, regardless of AI processing outcome.
- Failed AI jobs are **retryable** from a queue, not silently dropped.
- AI output is **validated** (schema + confidence checks) before any database insertion.
- Unknown skills detected by AI enter an **approval/review workflow** (Super Admin → Skill Aliases) rather than blindly creating unlimited new Skill records.

This pattern — synchronous core action, asynchronous intelligence enrichment with a visible status field — is used everywhere AI touches a user-facing action in this platform, not just job posting.

Observability

Every layer — including AI — is monitored, not just the web app

Error tracking Sentry across frontend and backend	Application visibility - Application logs - API monitoring	AI-specific visibility - AI processing logs - Failed job processing queue - Processing status per record (pending/processing/completed/failed)
Governance visibility - Audit logs - Database monitoring		

UI/UX Architecture

A light, professional, data-dense design language across every panel

Design language

Light theme

Clean & professional

Modern gov/BI dashboard

Responsive

Accessible

Minimal animation

Clear hierarchy

Light-theme-first by design decision — this is a government decision-support tool used in official, often print-adjacent contexts; it is not a dark-theme-first consumer product.

Common UI elements across panels

Tables · charts · maps · filters · search · cards · drill-down views

Responsive layout strategy

BREAKPOINT	LAYOUT
Desktop	Sidebar + Topbar + Main content
Tablet	Collapsible sidebar
Mobile	Bottom / slide navigation where appropriate

Government Dashboard Wireframe

The information architecture of the primary decision-support screen

CONCEPTUAL GOVERNMENT SKILL INTELLIGENCE DASHBOARD

Jobs Analysed

High Demand

Critical Gaps

Capacity

↓

District Demand Map

↓

SKILL	DEMAND	GROWTH	SCORE	STATUS
EV Battery Diagnostics	1,240	▲ 38%	88	SHORTAGE
Cloud DevOps	980	▲ 22%	81	SHORTAGE
Manual Bookkeeping	140	▼ 12%	24	OVERSUPPLY

Figures above are illustrative sample rows to demonstrate the dashboard layout, not live data.

↓

Critical Shortages

Emerging Skills

↓

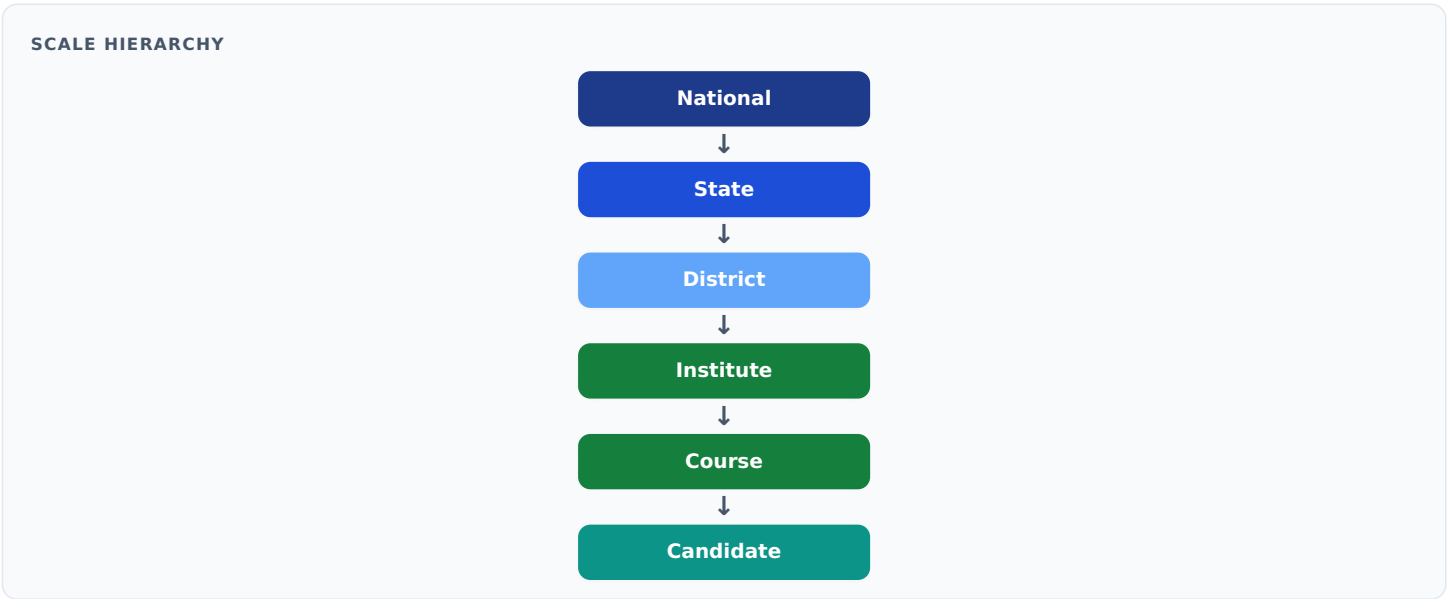
AI Recommendations

Forecast

This wireframe is conceptual — it defines the information architecture of the dashboard (what sections exist and in what order), not final pixel styling.

National Scale Architecture

The same model, from one district to the whole country



What makes this architecture scale, without over-engineering the MVP

Modular backend • API-driven architecture • Clear service boundaries inside one deployable app at MVP	Data scale • Indexed MongoDB collections • Caching where required • Data partitioning strategy by district/state as volume grows	AI scale • Asynchronous AI processing • Background jobs / queues • Scalable AI service, future Python microservice if load requires it
Operations at scale • Analytics pipeline • Observability (see earlier section)		

The same panel/permission/data model that works for one district works unchanged for every state and the whole country — scale is handled by adding rows (more districts, more institutes) and read-side optimisation (indexing, caching), not by redesigning the schema or the roles.

Important Architectural Principles

Fifteen rules that every phase of the build must respect

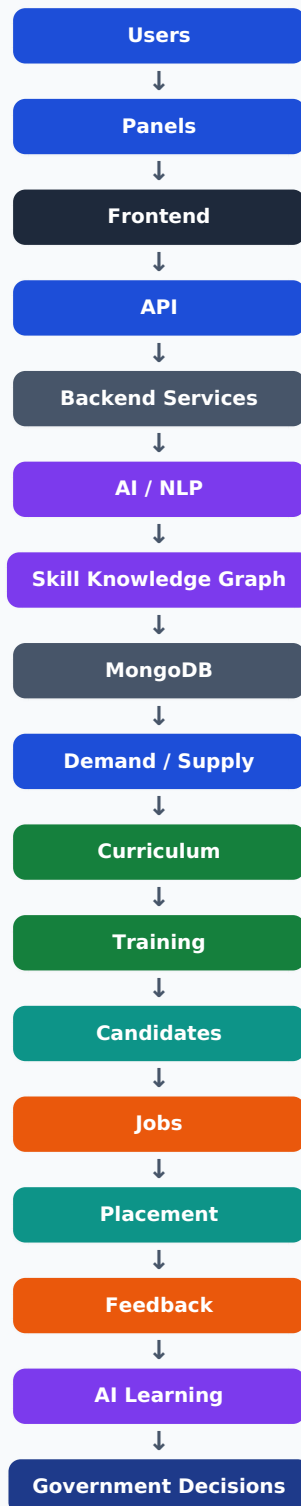
Fifteen non-negotiable principles that govern every implementation decision in this document.

#	PRINCIPLE
1	Existing Job Portal remains intact.
2	New modules extend existing entities.
3	Avoid duplicate User/Company/Job entities.
4	AI is modular and replaceable.
5	AI failure must not break core job posting.
6	Human validation remains important.
7	Predictions must be distinguishable from actual data.
8	Skill ontology must be controlled.
9	Data quality is critical.
10	Government decisions should be explainable.
11	Role-based access must be enforced.
12	Build incrementally.
13	Test every phase.
14	Do not build all AI functionality at once.
15	Keep MVP simple and scalable.

Final Master Architecture

Everything in this document, in one diagram

THE COMPLETE PLATFORM, TOP TO BOTTOM



The operating cycle beneath every layer

EIGHT-STEP CYCLE, REPEATED CONTINUOUSLY



This is the platform, end to end: every panel, every service, every AI job exists to move one more unit of data through this eight-step cycle a little faster and a little more accurately than the manual process it replaces.

Final Quality Check

Verifying nothing from the brief was dropped

CHECK	
✓	No major feature from the source concept is omitted.
✓	All eight panels are clearly defined with non-overlapping responsibility.
✓	Each role has clear, matrix-defined permissions.
✓	Existing Job Portal integration and rationale are explained.
✓	AI and NLP responsibilities are each clearly separated (12 distinct AI jobs).
✓	Forecasting is separated from simple descriptive analytics.
✓	Skill ontology and normalization are explained with a worked example.
✓	Demand/supply mismatch scoring is explained with its formula.
✓	Curriculum gap analysis is explained with a worked example.
✓	Placement feedback loop is explained end to end.
✓	District planning and the digital twin are explained.
✓	The What-If simulator is explained and its outputs are labelled as estimates.
✓	Database entities, relationships, APIs and technology stack are all shown.
✓	Security, reliability, observability and UI/UX are all addressed.
✓	Implementation is phased (16 phases) with MVP vs. advanced clearly separated.
✓	Actual, predicted and simulated data are distinguished throughout.

No uploaded source PDF (“Skill_Demand_Intelligence_Platform.pdf”) was present in this session's files. This document was built directly from the fully detailed brief supplied in the request itself, which already specified every concept, panel, workflow, and example listed above — nothing in that brief has been dropped. If a separate source PDF exists, share it and this document can be reconciled against it line by line.